Privacy-Preserving Facial Verification System

Data Privacy Course Project

Consent · Encryption · k-Anonymity · Differential Privacy

The Problem

Coded Bias (2020) — Documentary Findings

- MIT researcher Joy Buolamwini found commercial facial recognition failed on darker-skinned women up to 34% of the time — vs <1% for lighter-skinned men
- Cause: training datasets were overwhelmingly white and male
- Same 'accurate' system — dramatically different outcomes per demographic group
- Used in hiring, policing, building access — errors have real-world consequences

The Three Root Problems

Biometric Data is Permanent

Unlike passwords, you can't change your face after a breach

Scope Creep to Surveillance

Systems built for limited use get repurposed for tracking

Biased Training Data = Biased Output

Garbage in, garbage out — but the garbage harms specific people

Recognition vs Verification

The most important design decision in this project

Facial Recognition

- Input: unknown face
- Searches entire database
- Output: 'This is Alice'
- Enables mass surveillance
- Can identify anyone, anywhere
- Like searching a phonebook

Facial Verification ✓ (this system)

- Input: face + claimed identity
- Compares against ONE template only
- Output: 'Yes, this is Alice'
- Cannot identify unknown people
- Only works for enrolled users
- Like checking your own ID

System Architecture

1 Image In

Browser captures webcam frame
Encoded as base64 in form POST
Raw image bytes — never touched disk

→

2 Face Embedding

InsightFace detects + aligns face
ArcFace → 512-d vector
L2-normalised (unit vector)

→

3 Encrypt + Store

JSON payload (shape, dtype, data)
Fernet AES-128-CBC + HMAC
Written to data/{user_id}.enc

→

4 Verify

Load + decrypt stored template
Cosine similarity (dot product)
Pass if score ≥ 0.85 threshold

Privacy Guarantee Applied at Each Stage

- ✓ Image never leaves memory → no raw photo storage
- ✓ Template stored encrypted → unreadable without key
- ✓ One-to-one match only → identification impossible
- ✓ Audit outputs use k-anonymity + differential privacy

Privacy by Design — 7 Layers

L1 Data Minimisation

Raw images discarded immediately. Only a 512-number vector is kept.

L2 Purpose Limitation

No search endpoint. Identification is architecturally impossible.

L3 Consent & Autonomy

Explicit opt-in. User chooses their ID. Delete anytime, immediate effect.

L4 Encryption at Rest

Fernet AES-128-CBC + HMAC. Unreadable without the key.

L5 Access Control

Files outside web root. Key in env var. .gitignore excludes data/.

L6 Retention Policy

Templates auto-expire after 90 days. User-triggered deletion is permanent.

L7 Formal Privacy (DP)

k-Anonymity + Differential Privacy on all audit outputs.

k-Anonymity

Protecting identity in aggregate outputs

Definition

A dataset satisfies k-anonymity if every record is indistinguishable from at least k−1 other records with respect to quasi-identifiers (QIs).

Quasi-identifiers: attributes that are harmless alone but identifying in combination (e.g. age group + region + gender).

Sweeney (2002): 87% of Americans can be uniquely identified from just ZIP code + date of birth + gender.

Example: Audit Output (k = 3)

Age Group	Region	Records	Published?
20–30	North	6	✓ Yes
30–40	South	4	✓ Yes
40–50	East	2	× Suppressed
50–60	West	1	× Suppressed

Limitations

- Homogeneity attack: all k records share the same sensitive value → value revealed
- Background knowledge: external info can narrow down which record is 'yours'
- Doesn't compose: joining multiple k-anon releases can re-identify individuals
- → This is why we also apply Differential Privacy

Differential Privacy

Mathematical guarantee: individual contributions are undetectable

ε-Differential Privacy

For any two datasets D, D' differing by one record,
and any output S:

$$\Pr[M(D) \in S] \leq e^{\varepsilon} \times \Pr[M(D') \in S]$$

Smaller ε = stronger privacy = more noise added

Laplace Mechanism

$$M(x) = f(x) + \text{Lap}(\Delta f / \varepsilon)$$

f(x) = true query result
Δf = global sensitivity (max change from 1 record)
ε = privacy budget

Count query: Δf = 1 → noise scale = 1/ε

The ε Tradeoff

ε = 0.1	Very strong privacy	Numbers may be off by ±10
ε = 1.0	Standard (this project)	Off by ±1–3 ✓
ε = 10.0	Weak privacy	Near-exact but little protection

k-Anonymity + Differential Privacy = Defence in Depth

Threat Model

Database Theft	Attack: Attacker steals data/ folder	✓ Fernet encryption — files unreadable without key in env var
Scope Creep	Attack: System connected to public cameras	✓ No identification endpoint exists — would require new code
Re-identification	Attack: Audit logs reveal individual behaviour	✓ k-anonymity + differential privacy on all released statistics
Government Compulsion	Attack: Legal access demanded to stored data	✓ Minimal retention (90 days), immediate deletion right, encrypted files
Photo-based Spoofing	Attack: Photo presented to camera instead of live face	✓ Limitation: no liveness detection — future work

Technical Implementation

Technology	Role	Why this choice
Flask 3.x	Web framework	Lightweight, Python-native routes
InsightFace + ArcFace	Face detection + embedding	ONNX backend — no TensorFlow/PyTorch
OpenCV	Image decoding	Handles JPEG, PNG, WebP — any format
cryptography (Fernet)	AES-128-CBC + HMAC encryption	Industry standard authenticated encryption
NumPy	Cosine similarity + Laplace noise	Dot product on L2-normalised vectors
pytest (56 tests)	Test suite	Mocking, isolation, coverage across all modules

Fairness and Bias Mitigation

The Core Issue

A system that is 90% accurate overall but 60% accurate for one group is NOT a good system. It is a discriminatory system.

Aggregate accuracy hides per-group harm. This is what Coded Bias documented.

Two Key Metrics

FAR — False Acceptance Rate

Definition: Impostor accepted as genuine
Risk: Security risk

FRR — False Rejection Rate

Definition: Genuine user rejected
Risk: Fairness risk: if higher for one group → discrimination

What This Project Does

- ArcFace model — trained on diverse datasets, better baseline fairness than older approaches
- FAR and FRR measured separately per demographic group — not just in aggregate
- Configurable threshold — adjust per deployment context and risk tolerance
- Honest about limitations — no pre-trained model is fully bias-free; we measure and document

Test Suite — 56 Passed, 0 Failed

Module	Tests	Result	Coverage focus
embeddings.py	13	9 pass / 4 skip*	Image decode, L2 normalise, no-face handling
storage.py	12	12 pass	Encrypt/decrypt roundtrip, deletion, path sanitisation
enrollment.py	10	10 pass	Consent gate, duplicate detection, face failure
verification.py	10	10 pass	Cosine similarity, threshold, unknown users
privacy.py	13	13 pass	Laplace noise, private count/mean, k-anonymity

* 4 real-face tests skipped — activate when a face image fixture is provided

Test Isolation Strategy

Each test gets its own tmp_path directory + fresh Fernet key. Enrollment/verification tests mock generate_embedding() — fast, deterministic, no face needed.

Conclusion

1. Verification ≠ Recognition

One architectural decision eliminates the surveillance use case entirely. No search endpoint = no mass identification, by construction.

2. Privacy as Architecture

k-Anonymity and Differential Privacy are mathematical guarantees, not promises. You cannot override a proof with an organisational directive.

3. Redefining Success

The real lesson from Coded Bias: if you define success as 'highest accuracy,' you build a discriminatory system. Define success as 'equal accuracy and respect for privacy' and every design decision changes.

Built with: Flask · InsightFace (ArcFace) · Fernet encryption · k-Anonymity · Differential Privacy (Laplace) · 56 automated tests